

Graphs

Wednesday, June 10, 2026 6:30 PM

A Graph is composed of a set of vertices(**V**) and a set of edges(**E**). The vertices are connected with each other through edges.

- The limitation of tree is, it can only represent hierarchical data. For situations where nodes or vertices are randomly connected with each other, we use Graph.
- Example situations where we use graph data structure are, a social network, a computer network, a network of locations used in GPS and many more examples where different nodes or vertices are connected without any hierarchic or constraint on structure.

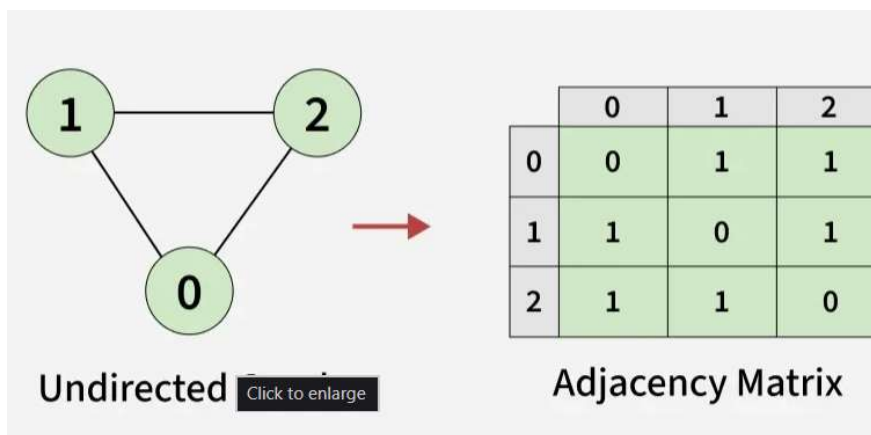
Representations of Graph

Here are the two most common ways to represent a graph : For simplicity, we are going to consider only **unweighted graphs** in this post.

1. Adjacency Matrix
2. Adjacency List

An adjacency matrix is a way of representing a graph as a boolean matrix of (0's and 1's). Let's assume there are n vertices in the graph So, create a 2D matrix $\text{adjMat}[n][n]$ having dimension $n \times n$.

- If there is an edge from vertex i to j , mark $\text{adjMat}[i][j]$ as 1.
- If there is no edge from vertex i to j , mark $\text{adjMat}[i][j]$ as 0.



```
def createGraph(V, edges):
    mat = [[0 for _ in range(V)] for _ in range(V)]

    # Add each edge to the adjacency matrix
    for it in edges:
        u = it[0]
        v = it[1]
        mat[u][v] = 1

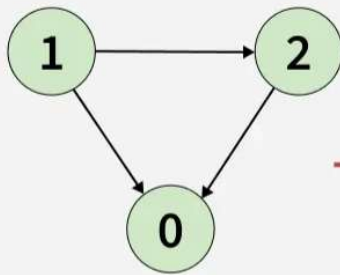
        # since the graph is undirected
        mat[v][u] = 1
    return mat

if __name__ == "__main__":
    V = 3

    # List of edges (u, v)
    edges = [[0, 1], [0, 2], [1, 2]]

    # Build the graph using edges
    mat = createGraph(V, edges)

    print("Adjacency Matrix Representation:")
    for i in range(V):
        for j in range(V):
            print(mat[i][j], end=" ")
        print()
```



Directed Graph

	0	1	2
0	0	0	0
1	1	0	1
2	1	0	0

Adjacency Matrix

```

def createGraph(V, edges):
    mat = [[0 for _ in range(V)] for _ in range(V)]

    # Add each edge to the adjacency matrix
    for it in edges:
        u = it[0]
        v = it[1]
        mat[u][v] = 1

    return mat
  
```

```

if __name__ == "__main__":
    V = 3
  
```

```

# List of edges (u, v)
edges = [[1, 0], [2, 0], [1, 2]]
  
```

```

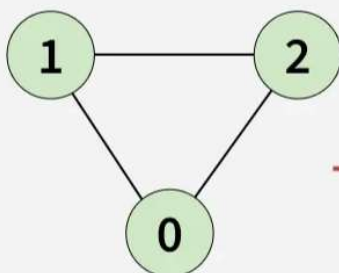
# Build the graph using edges
mat = createGraph(V, edges)
  
```

```

print("Adjacency Matrix Representation:")
for i in range(V):
    for j in range(V):
        print(mat[i][j], end=" ")
    print()
  
```

An array of Lists is used to store edges between two vertices. The size of array is equal to the number of vertices (i.e, n). Each index in this array represents a specific vertex in the graph. The entry at the index i of the array contains a linked list containing the vertices that are adjacent to vertex i. Let's assume there are n vertices in the graph So, create an array of list of size n as adjList[n].

- o adjList[0] will have all the nodes which are connected (neighbour) to vertex 0.
- o adjList[1] will have all the nodes which are connected (neighbour) to vertex 1 and so on.



Undirected Graph

Vertex	Neighbour	
0	→ 1	2
1	→ 0	2
2	→ 0	1

Adjacency List

```

def createGraph(V, edges):
    adj = [[] for _ in range(V)]
  
```

```

# Add each edge to the adjacency list
for it in edges:
    u = it[0]
    v = it[1]
    adj[u].append(v)
  
```

```

    # since the graph is undirected
    adj[v].append(u)
    return adj

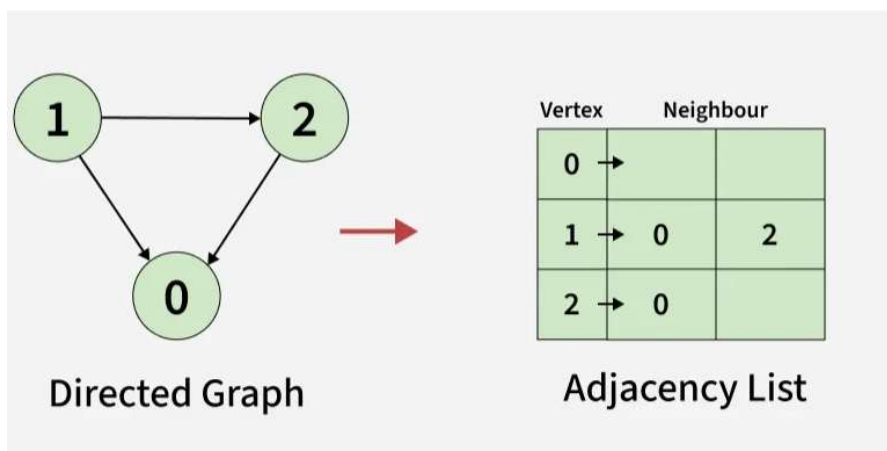
if __name__ == "__main__":
    V = 3

    # List of edges (u, v)
    edges = [[0, 1], [0, 2], [1, 2]]

    # Build the graph using edges
    adj = createGraph(V, edges)

    print("Adjacency List Representation:")
    for i in range(V):
        # Print the vertex
        print(f"{i}:", end=" ")
        for j in adj[i]:
            # Print its adjacent
            print(j, end=" ")
        print()

```



```

def createGraph(V, edges):
    adj = [[] for _ in range(V)]

    # Add each edge to the adjacency list
    for it in edges:
        u = it[0]
        v = it[1]
        adj[u].append(v)

    return adj

if __name__ == "__main__":
    V = 3

    # List of edges (u, v)
    edges = [[1, 0], [1, 2], [2, 0]]

    # Build the graph using edges
    adj = createGraph(V, edges)

    print("Adjacency List Representation:")
    for i in range(V):
        # Print the vertex
        print(f"{i}:", end=" ")
        for j in adj[i]:
            # Print its adjacent
            print(j, end=" ")
        print()

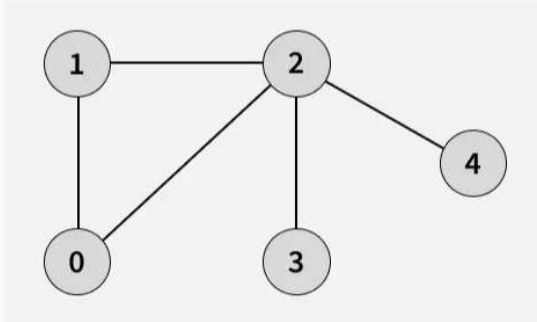
```

BFS

Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level. First, it visits all nodes directly adjacent to the source. Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

- Popular graph algorithms like [Dijkstra's shortest path](#), [Kahn's Algorithm](#), and [Prim's algorithm](#) are based on BFS.
- BFS itself can be used to detect cycle in a directed and undirected graph, find shortest path in an unweighted graph and many more problems.

Input: `adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]`



Output: `[0, 1, 2, 3, 4]`

Explanation: Starting from 0, the BFS traversal proceeds as follows:

Visit 0 - Print: 0

Visit 1 (neighbor of 0) - Print: 1

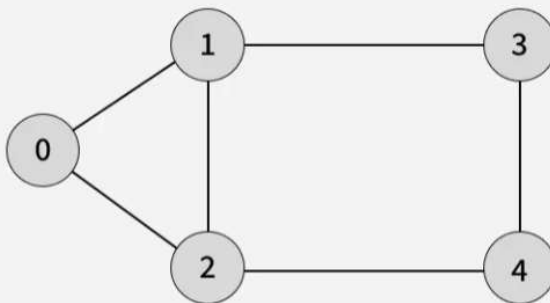
Visit 2 (next neighbor of 0) - Print: 2

Visit 3 (first neighbor of 2 that hasn't been visited yet) - Print: 3

Visit 4 (next neighbor of 2) - Print: 4

Input: `adj[][] = [[2, 3], [2], [0, 1], [0], [5], [4]]`

Given the following graph, find its BFS traversal



```
from collections import deque
```

```
# BFS for single connected component
```

```
def bfs(adj):
```

```
    V = len(adj)
```

```
    visited = [False] * V
```

```
    res = []
```

```
    src = 0
```

```
    q = deque()
```

```
    visited[src] = True
```

```
    q.append(src)
```

```
    while q:
```

```
        curr = q.popleft()
```

```
        res.append(curr)
```

```
        # visit all the unvisited
```

```
        # neighbours of current node
```

```
        for x in adj[curr]:
```

```
            if not visited[x]:
```

```
                visited[x] = True
```

```
                q.append(x)
```

```

    return res

def addEdge(adj, u, v):
    adj[u].append(v)
    adj[v].append(u)

if __name__ == "__main__":
    V = 5
    adj = []

    # creating adjacency list
    for i in range(V):
        adj.append([])

    addEdge(adj, 1, 2)
    addEdge(adj, 1, 0)
    addEdge(adj, 2, 0)
    addEdge(adj, 2, 3)
    addEdge(adj, 2, 4)

    res = bfs(adj)

    for node in res:
        print(node, end=" ")

```

Time Complexity: $O(V + E)$, BFS explores all the vertices and edges in the graph. It visits every vertex and edge only once.

Auxiliary Space: $O(V)$, Using a queue to keep track of the vertices that need to be visited.

BFS of a Disconnected Undirected Graph:

In a disconnected graph, some vertices may not be reachable from a single source. To ensure all vertices are visited in BFS traversal, we iterate through each vertex, and if any vertex is unvisited, we perform a BFS starting from that vertex being the source. This way, BFS explores every connected component of the graph.

```

from collections import deque

# BFS for a single connected component
def bfsConnected(adj, src, visited, res):
    q = deque()
    visited[src] = True
    q.append(src)

    while q:
        curr = q.popleft()
        res.append(curr)

        # visit all the unvisited
        # neighbours of current node
        for x in adj[curr]:
            if not visited[x]:
                visited[x] = True
                q.append(x)

# BFS for all components (handles disconnected graphs)
def bfs(adj):
    V = len(adj)
    visited = [False] * V
    res = []

    for i in range(V):
        if not visited[i]:
            bfsConnected(adj, i, visited, res)

    return res

def addEdge(adj, u, v):
    adj[u].append(v)
    adj[v].append(u)

if __name__ == "__main__":
    V = 6
    adj = []

    # creating adjacency list
    for i in range(V):
        adj.append([])

```

```

addEdge(adj, 1, 2)
addEdge(adj, 2, 0)
addEdge(adj, 0, 3)
addEdge(adj, 4, 5)

```

```
res = bfs(adj)
```

```

for node in res:
    print(node, end=" ")

```

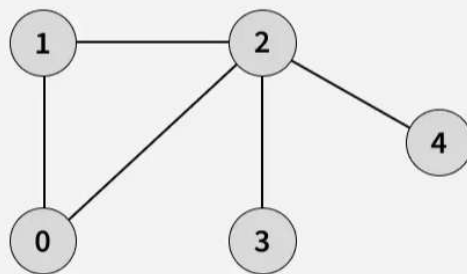
Time Complexity: $O(V + E)$, The for loop ensures BFS starts from every unvisited vertex to cover all components, but the visited array ensures each vertex and edge is processed only once, keeping the total time complexity to be linear.

Auxiliary Space: $O(V)$, using a queue to keep track of the vertices that need to be visited.

DFS

In Depth First Search (or DFS) for a graph, we traverse all adjacent vertices one by one. When we traverse an adjacent vertex, we completely finish the traversal of all vertices reachable through that adjacent vertex.

Input: adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]



Output: [0, 1, 2, 3, 4]

Explanation: The source vertex is 0. We visit it first, then we visit its adjacent.

Start at 0: Mark as visited. Print 0.

Move to 1: Mark as visited. Print 1.

Move to 2: Mark as visited. Print 2.

Move to 3: Mark as visited. Print 3.(backtrack to 2)

Move to 4: Mark as visited. Print 4(backtrack to 2, then backtrack to 1, then to 0).

Note that there can be more than one DFS Traversals of a Graph. For example, after 1, we may pick adjacent 2 instead of 0 and get a different DFS.

Input: adj[][] = [[2, 3], [2], [0, 1], [0], [5], [4]]

DFS from a Given Source of Graph:

Depth First Search (DFS) starts from a given source vertex and explores one path as deeply as possible. When it reaches a vertex with no unvisited neighbors, it backtracks to the previous vertex to explore other unvisited paths. This continues until all vertices reachable from the source are visited.

In a graph, there might be loops. So we use an extra visited array to make sure that we do not process a vertex again.

```
def dfsRec(adj, visited, s, res):
```

```
    visited[s] = True
```

```
    res.append(s)
```

```
    # Recursively visit all adjacent vertices
```

```
    # that are not visited yet
```

```
    for i in adj[s]:
```

```
        if not visited[i]:
```

```
            dfsRec(adj, visited, i, res)
```

```
def dfs(adj):
```

```
    visited = [False] * len(adj)
```

```
    res = []
```

```
    dfsRec(adj, visited, 0, res)
```

```
    return res
```

```
def addEdge(adj, u, v):
```

```
    adj[u].append(v)
```

```
    adj[v].append(u)
```

```
if __name__ == "__main__":
```

```

V = 5
adj = []

# creating adjacency list
for i in range(V):
    adj.append([])

addEdge(adj, 1, 2)
addEdge(adj, 1, 0)
addEdge(adj, 2, 0)
addEdge(adj, 2, 3)
addEdge(adj, 2, 4)

# Perform DFS starting from vertex 0
res = dfs(adj)

for node in res:
    print(node, end=" ")

```

Time complexity: $O(V + E)$, where V is the number of vertices and E is the number of edges in the graph.

Auxiliary Space: $O(V + E)$, since an extra visited array of size V is required, And stack size for recursive calls to dfsRec function.

DFS of a Disconnected Graph:

In a disconnected graph, some vertices may not be reachable from a single source. To ensure all vertices are visited in DFS traversal, we iterate through each vertex, and if a vertex is unvisited, we perform a DFS starting from that vertex being the source. This way, DFS explores every connected component of the graph.

```

from collections import defaultdict

```

```

def dfsRec(adj, visited, s, res):
    visited[s] = True
    res.append(s)

    # Recursively visit all adjacent
    # vertices that are not visited yet
    for i in adj[s]:
        if not visited[i]:
            dfsRec(adj, visited, i, res)

```

```

def dfs(adj):
    visited = [False] * len(adj)
    res = []
    # Loop through all vertices to
    # handle disconnected graph
    for i in range(len(adj)):
        if not visited[i]:
            dfsRec(adj, visited, i, res)
    return res

```

```

def addEdge(adj, u, v):
    adj[u].append(v)
    adj[v].append(u)

```

```

if __name__ == "__main__":
    V = 6
    adj = []

    # creating adjacency list
    for i in range(V):
        adj.append([])

    addEdge(adj, 1, 2)
    addEdge(adj, 2, 0)
    addEdge(adj, 0, 3)
    addEdge(adj, 5, 4)

    # Perform DFS
    res = dfs(adj)

    print(*res)

```

Time complexity: $O(V + E)$. We visit every vertex at most once and every edge is traversed at most once (in directed) and twice in undirected.

Auxiliary Space: $O(V + E)$, since an extra visited array of size V is required, and stack size for recursive calls of dfs function.

Esistono diverse tipologie di archi:

- Tree: archi dell'albero della visita in profondita
- Backward: connettono un vertice ad un suo antenato
- Forward: connettono un vertice ad un suo discendente
- Cross: altri casi

ARTICULATION POINTS IN A GRAPH

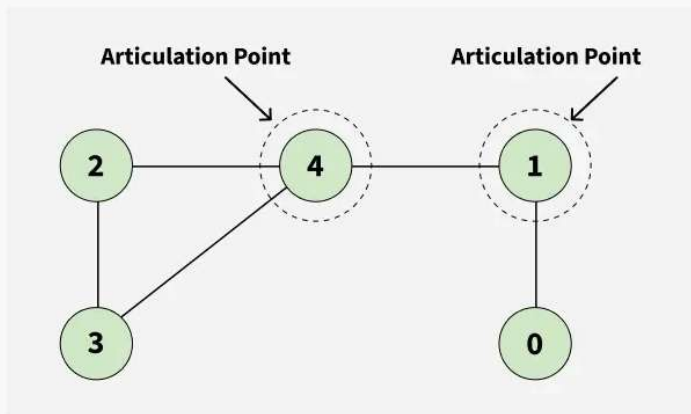
Given an **undirected** graph consisting of **V** vertices and **E** edges. The graph is represented as a 2D array **edges**[[i]], where each element **edges**[i] = [u, v] denotes an undirected edge between vertices u and v.

Find all the **articulation points** in the graph. If no such points exist, return {-1}.

An articulation point is a vertex whose removal, along with all its connected edges, increases the number of connected components in the graph.

Note: The graph may be disconnected, meaning it can consist of multiple connected components

Input: $V = 5$, $edges[[i]] = [[0, 1], [1, 4], [4, 3], [4, 2], [2, 3]]$



Output: [1, 4]

Explanation: Removing vertex 1 or 4 disconnects the graph, as illustrated below:

```
def constructadj(V, edges):
    # Builds adjacency list from edge list
    adj = [[] for _ in range(V)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    return adj

# Standard DFS to mark all reachable nodes
def dfs(node, adj, visited):
    # Standard DFS to mark all reachable nodes
    visited[node] = True

    for neighbor in adj[node]:
        if not visited[neighbor]:
            dfs(neighbor, adj, visited)

def articulationPoints(V, adj):
    res = []

    # Try removing each node one by one
    for i in range(V):
        visited = [False] * V
        visited[i] = True

        # count DFS calls from i's neighbors
        comp = 0
        for it in adj[i]:
            if comp > 1:
                break
            if not visited[it]:
                # explore connected part
                dfs(it, adj, visited)
            comp += 1
```



```

# if more than one component forms, it's an articulation point
if comp > 1:
    res.append(i)

if not res:
    return [-1]

return res

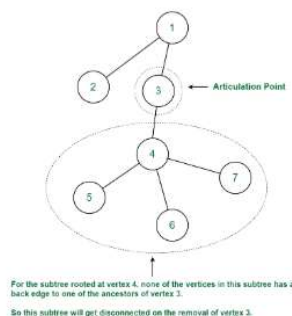
if __name__ == "__main__":
    V = 5
    edges = [[0, 1], [1, 4], [2, 3], [2, 4], [3, 4]]

    # Finds articulation points using naive DFS approach
    adj = constructadj(V, edges)
    ans = articulationPoints(V, adj)
    for it in ans:
        print(it, end=" ")

Time complexity: O(V * (V + E))
Space O(V)

```

The idea is to use DFS to track how each node connects to its ancestors using discovery time and low values. Here, discovery time records when a node is first visited, and the low value represents the earliest (smallest) discovery time reachable from that node, including via back edges. For every node, we try to determine whether its subtree has an alternative path to reach an ancestor. If such a path does not exist for a child subtree, then the current node becomes a critical connection point, and removing it would disconnect that subtree from the rest of the graph. By propagating these low values during DFS, we can efficiently identify all nodes whose removal increases the number of connected components, i.e., the articulation points.



For the vertex 3 (which is not the root), vertex 4 is the child of vertex 3. No vertex in the subtree rooted at vertex 4 has a back edge to one of the ancestors of vertex 3. Thus on removal of vertex 3 and its associated edges the graph will get disconnected or the number of components in the graph will increase as the subtree rooted at vertex 4 will form a separate component. Hence vertex 3 is an articulation point.

Step by Step implementation:

Maintain these arrays and integers and perform a DFS traversal.

- `disc[]`: Discovery time of each vertex during DFS.
- `low[]`: The lowest discovery time reachable from the subtree rooted at that vertex (via tree or back edges).
- `parent`: To keep track of each node's parent in the DFS tree.
- `visited[]`: To mark visited nodes.

Root Node Case:

- For the root node of DFS (i.e., `parent[u] == -1`), check how many child DFS calls it makes.
- If the root has two or more children, it is an articulation point

For any non-root node `u`, check all its adjacent nodes:

- If `v` is an unvisited child, recur for `v`, and after returning update `low[u] = min(low[u], low[v])`.
- If `low[v] >= disc[u]`, then `u` is an articulation point because `v` and its subtree cannot reach any ancestor of `u`, so removing `u` would disconnect `v`.

Back Edge Case

- If `v` is already visited and is not the parent of `u` then It's a back edge. Update `low[u] = min(low[u], disc[v])`
- This helps bubble up the lowest reachable ancestor through a back edge.

```

def constructAdj(V, edges):
    adj = [[] for _ in range(V)]

    for edge in edges:
        adj[edge[0]].append(edge[1])
        adj[edge[1]].append(edge[0])
    return adj

# Helper function to perform DFS and find articulation points
# using Tarjan's algorithm.
def findPoints(adj, u, visited, disc, low, time, parent, isAP):

    # Mark vertex u as visited and assign discovery
    # time and low value
    visited[u] = 1
    time[0] += 1
    disc[u] = low[u] = time[0]
    children = 0

    # Process all adjacent vertices of u
    for v in adj[u]:

        # If v is not visited, then recursively visit it
        if not visited[v]:
            children += 1
            findPoints(adj, v, visited, disc, low, time, u, isAP)

            # Check if the subtree rooted at v has a
            # connection to one of the ancestors of u
            low[u] = min(low[u], low[v])

            # If u is not a root and low[v] is greater than or equal to disc[u],
            # then u is an articulation point
            if parent != -1 and low[v] >= disc[u]:
                isAP[u] = 1

        # Update low value of u for back edge
        elif v != parent:
            low[u] = min(low[u], disc[v])

    # If u is root of DFS tree and has more than
    # one child, it is an articulation point
    if parent == -1 and children > 1:
        isAP[u] = 1

# Main function to find articulation points in the graph
def articulationPoints(V, adj):

    disc = [0] * V
    low = [0] * V
    visited = [0] * V
    isAP = [0] * V
    time = [0]

    # Run DFS from each vertex if not
    # already visited (to handle disconnected graphs)
    for u in range(V):
        if not visited[u]:
            findPoints(adj, u, visited, disc, low, time, -1, isAP)

    # Collect all vertices that are articulation points
    result = [u for u in range(V) if isAP[u]]

    # If no articulation points are found, return list containing -1
    return result if result else [-1]

if __name__ == "__main__":
    V = 5
    edges = [[0, 1], [1, 4], [2, 3], [2, 4], [3, 4]]

    adj = constructAdj(V, edges)
    ans = articulationPoints(V, adj)

    for u in ans:
        print(u, end=' ')
    print()

```

Time Complexity $O(V + E)$
Space $O(V)$

Iterative approach on Tarjan's algorithm

```
def articulation_points(V, adj):

    # discovery times
    disc = [0] * V

    # low values
    low = [0] * V
    parent = [-1] * V
    children_count = [0] * V
    visited = [False] * V
    ap = [False] * V

    # Run iterative DFS for each component
    for i in range(V):
        if not visited[i]:
            tarjan(i, adj, visited, disc, low, parent, children_count, ap)

    # Collect articulation points
    result = [i for i in range(V) if ap[i]]
    return result if result else [-1]

def tarjan(start, adj, visited, disc, low, parent, children_count, ap):

    # Stack stores {node, neighborIndex, state}
    # state 0 = entering, 1 = exiting
    stack = [[start, 0, 0]]
    time = 0

    while stack:
        u, idx, state = stack.pop()

        if state == 0:

            # First visit: mark discovery and low
            if not visited[u]:
                visited[u] = True
                time += 1
                disc[u] = low[u] = time

            # Process neighbors
            if idx < len(adj[u]):
                v = adj[u][idx]

                # Push current node with next neighbor
                stack.append([u, idx + 1, 0])

                # Visit child
                if not visited[v]:
                    parent[v] = u
                    children_count[u] += 1
                    stack.append([v, 0, 0])

                # Back edge
                elif v != parent[u]:
                    low[u] = min(low[u], disc[v])
            else:
                # Finished neighbors, push exit state
                stack.append([u, 0, 1])

        else:

            # Update parent low and mark articulation
            if stack:
                p = stack[-1][0]
                low[p] = min(low[p], low[u])
                if parent[p] != -1 and low[u] >= disc[p]:
                    ap[p] = True

            # Root articulation check
            else:
                if children_count[start] > 1:
                    ap[start] = True

if __name__ == "__main__":
    V = 5

    # Build adjacency list
```

```

adj = [[] for _ in range(V)]
edges = [(0, 1), (1, 4), (2, 3), (2, 4), (3, 4)]
for u, v in edges:
    adj[u].append(v)
    adj[v].append(u)

print(*articulation_points(V, adj))

```

Time Complexity $O(V + E)$

Space $O(V)$

BRIDGES IN A GRAPH

An edge in an undirected connected graph is a bridge if removing it disconnects the graph. For a disconnected undirected graph, the definition is similar, a bridge is an edge removal that increases the number of disconnected components.

Naïve approach

- For every edge (u, v), do the following:
 - o Remove (u, v) from the graph
 - o See if the graph remains connected (either uses BFS or DFS)
 - o Add (u, v) back to the graph.

Time Complexity: $O(E*(V+E))$ for a graph represented by an adjacency list.

Auxiliary Space: $O(V+E)$

Tarjan Algorithm

Before heading towards the approach understand which edge is termed as bridge. Suppose there exists an edge from u -> v, now after removal of this edge if v can't be reached by any other edges then u -> v edge is bridge. Our approach is based on this intuition, so take time and grasp it.

ALGORITHM: -

To implement this algorithm, we need the following data structures -

- visited[] = to keep track of the visited vertices to implement DFS
- disc[] = to keep track when for the first time that particular vertex is reached
- low[] = to keep track of the lowest possible time by which we can reach that vertex 'other than parent' so that if edge from parent is removed can the particular node can be reached other than parent.

We will traverse the graph using DFS traversal but with slight modifications i.e. while traversing we will keep track of the parent node by which the particular node is reached because we will update the $low[node] = \min(low[all \text{ it's adjacent node except parent}])$ hence we need to keep track of the parent.

While traversing adjacent nodes let 'v' of a particular node let 'u', then 3 cases arise -

1. v is parent of u then,
 - skip that iteration.
2. v is visited then,
 - update the low of u i.e. $low[u] = \min(low[u], disc[v])$ this arises when a node can be visited by more than one node, but low is to keep track of the lowest possible time so we will update it.
3. v is not visited then,
 - call the DFS to traverse ahead
 - now update the $low[u] = \min(low[u], low[v])$ as we know v can't be parent cause we have handled that case first.
 - now check **if (low[v] > disc[u])** i.e. the lowest possible to time to reach 'v' is greater than 'u' this means we can't reach 'v' without 'u' so the edge u -> v is a bridge.

Python program to find bridges in a given undirected graph

#Complexity : $O(V+E)$

from collections import defaultdict

#This class represents an undirected graph using adjacency list representation

class Graph:

```

def __init__(self,vertices):
    self.V= vertices #No. of vertices
    self.graph = defaultdict(list) # default dictionary to store graph
    self.Time = 0

```

function to add an edge to graph

```

def addEdge(self,u,v):
    self.graph[u].append(v)
    self.graph[v].append(u)

```

```

"""A recursive function that finds and prints bridges
using DFS traversal
u --> The vertex to be visited next
visited[] --> keeps track of visited vertices
disc[] --> Stores discovery times of visited vertices
parent[] --> Stores parent vertices in DFS tree"""

```

```

def bridgeUtil(self,u, visited, parent, low, disc):

    # Mark the current node as visited and print it
    visited[u]= True

    # Initialize discovery time and low value
    disc[u] = self.Time
    low[u] = self.Time
    self.Time += 1

    #Recur for all the vertices adjacent to this vertex
    for v in self.graph[u]:
        # If v is not visited yet, then make it a child of u
        # in DFS tree and recur for it
        if visited[v] == False :
            parent[v] = u
            self.bridgeUtil(v, visited, parent, low, disc)

        # Check if the subtree rooted with v has a connection to
        # one of the ancestors of u
        low[u] = min(low[u], low[v])

        ''' If the lowest vertex reachable from subtree
        under v is below u in DFS tree, then u-v is
        a bridge'''
        if low[v] > disc[u]:
            print ("%d %d" %(u,v))

    elif v != parent[u]: # Update low value of u for parent function calls.
        low[u] = min(low[u], disc[v])

# DFS based function to find all bridges. It uses recursive
# function bridgeUtil()
def bridge(self):

    # Mark all the vertices as not visited and Initialize parent and visited,
    # and ap(articulation point) arrays
    visited = [False] * (self.V)
    disc = [float("Inf")] * (self.V)
    low = [float("Inf")] * (self.V)
    parent = [-1] * (self.V)

    # Call the recursive helper function to find bridges
    # in DFS tree rooted with vertex 'i'
    for i in range(self.V):
        if visited[i] == False:
            self.bridgeUtil(i, visited, parent, low, disc)

# Create a graph given in the above diagram
g1 = Graph(5)
g1.addEdge(1, 0)
g1.addEdge(0, 2)
g1.addEdge(2, 1)
g1.addEdge(0, 3)
g1.addEdge(3, 4)

print ("Bridges in first graph ")
g1.bridge()

g2 = Graph(4)
g2.addEdge(0, 1)
g2.addEdge(1, 2)
g2.addEdge(2, 3)
print ("\nBridges in second graph ")
g2.bridge()

g3 = Graph (7)
g3.addEdge(0, 1)
g3.addEdge(1, 2)
g3.addEdge(2, 0)
g3.addEdge(1, 3)
g3.addEdge(1, 4)
g3.addEdge(1, 6)
g3.addEdge(3, 5)

```

```

g3.addEdge(4, 5)
print("\nBridges in third graph ")
g3.bridge()

```

#This code is contributed by Neelam Yadav
Time complexity $O(V + E)$
Space $O(V)$

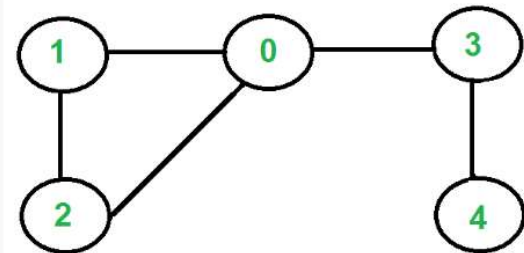
EULERIAN PATH AND CIRCUIT FOR UNDIRECTED GRAPH

Given an **undirected connected graph** with **v** nodes, and **e** edges, with adjacency list **adj**. We need to write a function that returns 2 if the graph contains an **eulerian circuit or cycle**, else if the graph contains an **eulerian path**, returns 1, otherwise, returns 0.

A graph is said to be Eulerian if it contains an **Eulerian Cycle**, a cycle that visits every edge exactly once and starts and ends at the same vertex.

If a graph contains an **Eulerian Path**, a path that visits every edge exactly once but starts and ends at different vertices, it is called **Semi-Eulerian**.

Input:



```

def dfs(node, adj, visited):
    visited[node] = True

    for neighbor in adj[node]:
        if not visited[neighbor]:
            dfs(neighbor, adj, visited)

# Function to check Eulerian Path or Circuit
def isEulerCircuit(v, adj):
    visited = [False] * v

    # Find first vertex with non-zero degree
    start = -1
    for i in range(v):
        if len(adj[i]) > 0:
            start = i
            break

    # No edges: graph is trivially Eulerian
    if start == -1:
        return 2

    # DFS from the first non-zero degree vertex
    dfs(start, adj, visited)

    # Check if all non-zero degree vertices are connected
    for i in range(v):
        if len(adj[i]) > 0 and not visited[i]:
            return 0 # Not connected

    # Count vertices with odd degree
    odd = 0
    for i in range(v):
        if len(adj[i]) % 2 != 0:
            odd += 1

    # Apply Eulerian rules
    if odd == 0:
        return 2
    elif odd == 2:
        return 1
    else:
        return 0

if __name__ == "__main__":
    v = 5
    adj = [
        [1, 2, 3],
        [0, 2],
        [1, 0],
        [0, 4],
        [3]
    ]

    print(isEulerCircuit(v, adj))

```

Time Complexity: $O(v + e)$, DFS traverses all vertices and edges to check connectivity and degree.

Space Complexity: $O(v + e)$, Space used for visited array and adjacency list to represent graph.

STRONGLY CONNECTED COMPONENTS

In a directed graph, a Strongly Connected Component is a subset of vertices where every vertex in the subset is reachable from every other vertex in the same subset by traversing the directed edges. Finding the **SCCs** of a graph can provide important insights into the structure and connectivity of the graph, with applications in various fields such as **social network analysis, web crawling, and network routing**.

[Expected Approach 1] Using Kosaraju's Algorithm

1. Performing Depth-First Search (DFS) on the Original Graph:

- We first do a DFS on the original graph and record the finish times of nodes (i.e., the time at which the DFS finishes exploring a node completely).

2. Performing DFS on the Transposed Graph:

- We then reverse the direction of all edges in the graph to create the transposed graph.
- Next, we perform a DFS on the transposed graph, considering nodes in decreasing order of their finish times recorded in the first phase.

- Each DFS traversal in this phase will give us one SCC.

```
from collections import defaultdict
```

```
class GfG:
```

```
    # Run a dfs on the original graph
```

```
    def DFS1(self, u, adj, visited, st):
```

```
        visited[u] = True
```

```
        for v in adj[u]:
```

```
            if not visited[v]:
```

```
                self.DFS1(v, adj, visited, st)
```

```
        st.append(u)
```

```
    # DFS on reversed graph to collect SCC
```

```
    def DFS2(self, u, revAdj, visited, scc):
```

```
        visited[u] = True
```

```
        scc.append(u)
```

```
        for v in revAdj[u]:
```

```
            if not visited[v]:
```

```
                self.DFS2(v, revAdj, visited, scc)
```

```
    def kosaraju(self, V, adj):
```

```
        visited = [False] * V
```

```
        st = []
```

```
        # Fill stack with finish time order
```

```
        for i in range(V):
```

```
            if not visited[i]:
```

```
                self.DFS1(i, adj, visited, st)
```

```
        # Reverse the graph
```

```
        revAdj = [[] for _ in range(V)]
```

```
        for u in range(V):
```

```
            for v in adj[u]:
```

```
                revAdj[v].append(u)
```

```
        # Process reversed graph in order of stack
```

```
        visited = [False] * V
```

```
        SCCs = []
```

```
        while st:
```

```
            u = st.pop()
```

```
            if not visited[u]:
```

```
                scc = []
```

```
                self.DFS2(u, revAdj, visited, scc)
```

```
                SCCs.append(scc)
```

```
        return SCCs
```

```
if __name__ == "__main__":
```

```
    obj = GfG()
```

```
    V = 5
```

```
    edges = [
```

```
        [1, 3], [1, 4], [2, 1], [3, 2], [4, 5]
```

```
    ]
```

```
    adj = [[] for _ in range(V + 1)]
```

```
    for u, v in edges:
```

```
        adj[u].append(v)
```

```
    SCCs = obj.kosaraju(V + 1, adj)
```

```
    print("Strongly Connected Components:")
```

```
    for i in range(len(SCCs) - 1):
```

```
        for node in SCCs[i]:
```

```
            print(node, end=" ")
```

```
        print()
```

Time Complexity - $O(V + E)$

Auxiliary Space - $O(V + E)$

[Expected Approach 2] Using Tarjan's Algorithm

Tarjan's Algorithm is more efficient because it finds SCCs in a single DFS pass using a stack and some additional bookkeeping:

1. **DFS Traversal:** During the DFS, maintain an index for each node and the smallest index (low-link value) that can be reached from the node.
2. **Stack:** Keep track of nodes currently in the recursion stack (part of the current SCC being explored).
3. **Identifying SCCs:** When a node's low-link value equals its index, it means we have

found an SCC. Pop all nodes from the stack until we reach the current node.

DAG

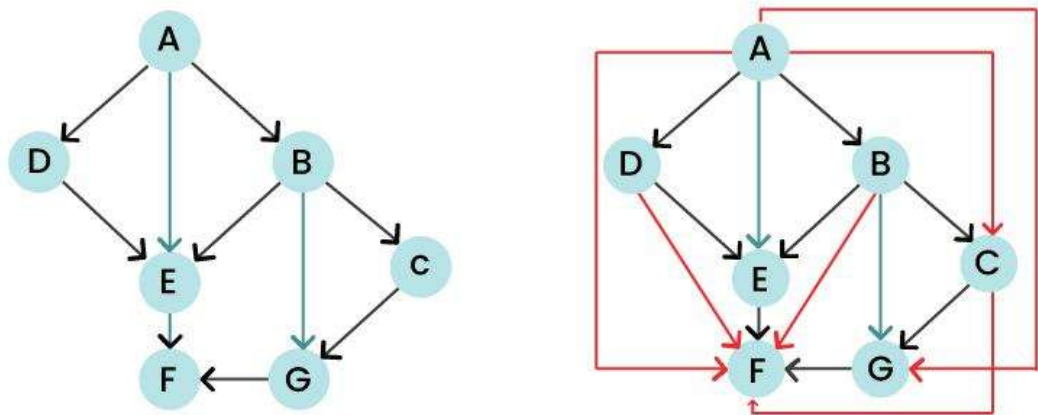
A **Directed Acyclic Graph (DAG)** is a directed graph that does not contain any cycles.

Properties of Directed Acyclic Graph DAG:

Directed Acyclic Graph (DAG) has different properties that make them usable in graph problems.

There are following properties of Directed Acyclic Graph (DAG):

- **Reachability Relation:** In DAG, we can determine if there is a reachability relation between two nodes. Node A is said to be reachable from node B if there exists a directed path that starts at node B and ends at node A. This implies that you can follow the direction of edges in the graph to get from B to A.
- **Transitive Closure:** The transitive closure of a directed graph is a new graph that represents all the direct and indirect relationships or connections between nodes in the original graph. In other words, it tells you which nodes can be reached from other nodes by following one or more directed edges.



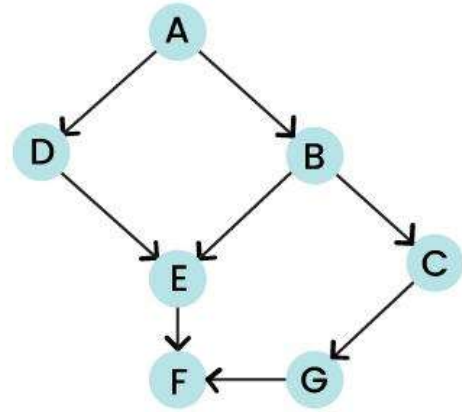
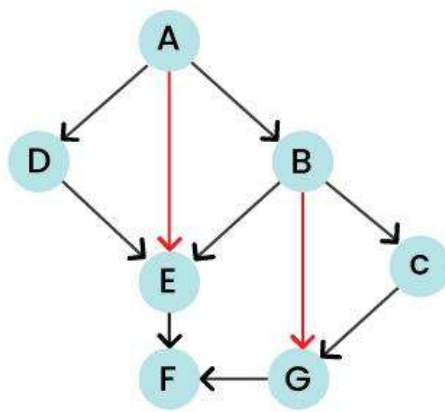
A Directed Acyclic Graph → Its Transitive Closure

Transitive closure of Directed Acyclic Graph



Transitive Closure of Directed Acyclic Graph

- **Transitive Reduction:** The transitive reduction of a directed graph is a new graph that retains only the essential, direct relationships between nodes, while removing any unnecessary indirect edges. In essence, it simplifies the graph by eliminating edges that can be inferred from the remaining edges.



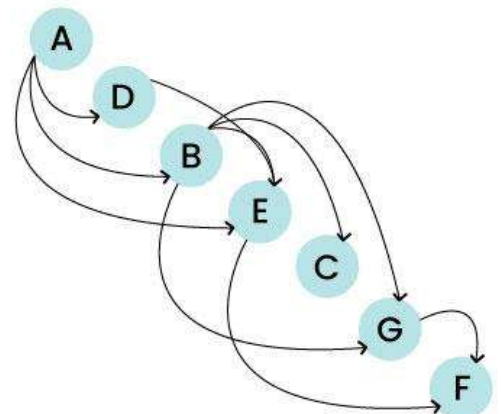
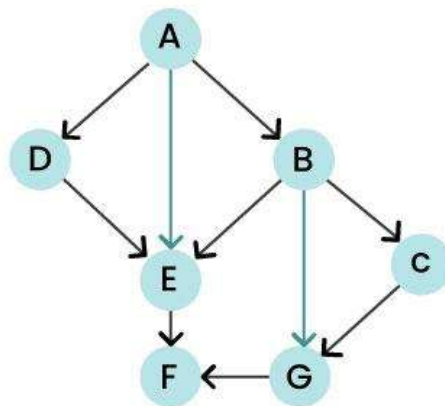
A Directed Acyclic Graph $\longrightarrow$ Its Transitive Reduction

Transitive Reduction of Directed Acyclic Graph



Transitive Reduction of Directed Acyclic Graph

- **Topological Ordering:** A DAG can be topologically sorted, which means you can linearly order its nodes in such a way that for all the edges, start node of the edge occurs earlier in the sequence. This property is useful for tasks like scheduling and dependency resolution.



A Directed Acyclic Graph $\longrightarrow$ Its Topological Sort/Order

Topological Ordering of Directed Acyclic Graph



Topological Sorting in Directed Acyclic Graphs (DAGs)

DAGs are graphs where each edge is directed and no cycle exists in the graph. Before understanding why Topological sort only exists for DAGs, let's first answer two questions:

- **Why Topological Sort is not possible for graphs with undirected edges or cycles?**

Undirected Graph:

Every edge $u \rightarrow v$ can be seen as both $u \rightarrow v$ and $v \rightarrow u$, so you can't decide which one should come first.

Cyclic Graph:

Let's say the graph contains the cycle, $u \rightarrow v \rightarrow w \rightarrow u$, then you'd need u before v , v before w , and w before u — impossible to satisfy.

- **Topological order may not be Unique:**

Topological sort represents all possible ordering satisfying the condition that if there is an edge between $u \rightarrow v$, u comes before v in the ordering. Any ordering that satisfies

this for all edges($u \rightarrow v$) is valid.

Topological Sorting Using DFS:

The main idea is to perform a Depth First Search (DFS) on the Directed Acyclic Graph (DAG) and, for each vertex, push it onto a stack only after visiting all its adjacent vertices. This ensures that every vertex appears after all its neighboring vertices.

Finally, reversing the stack (or popping elements from it) gives the topological ordering of the graph.

Refer to [this article](#) to read more.

Topological Sorting Using BFS:

Topological Sort using BFS (Kahn's Algorithm) works by repeatedly selecting vertices with in-degree zero (no dependencies), adding them to the result, and reducing the in-degree of their adjacent vertices. This process continues until all vertices are processed, producing a valid linear ordering of a DAG.

We can go through all possible ordering via backtracking, the algorithm steps are as follows:

3. Initialize all vertices as unvisited.
4. Now choose vertex which is unvisited and has zero in-degree and decrease in-degree of all those vertices by 1 (corresponding to removing edges) now add this vertex to result and call the recursive function again and backtrack.
5. After returning from function reset values of visited, result and in-degree for enumeration of other possibilities.

class to represent a graph object
class Graph:

```
# Constructor
def __init__(self, edges, N):

    # A List of Lists to represent an adjacency list
    self.adjList = [[] for _ in range(N)]

    # stores in-degree of a vertex
    # initialize in-degree of each vertex by 0
    self.indegree = [0] * N

    # add edges to the undirected graph
    for (src, dest) in edges:

        # add an edge from source to destination
        self.adjList[src].append(dest)

        # increment in-degree of destination vertex by 1
        self.indegree[dest] = self.indegree[dest] + 1

# Recursive function to find
# all topological orderings of a given DAG
def findAllTopologicalOrders(graph, path, discovered, N):

    # do for every vertex
    for v in range(N):

        # proceed only if in-degree of current node is 0 and
        # current node is not processed yet
        if graph.indegree[v] == 0 and not discovered[v]:

            # for every adjacent vertex u of v,
            # reduce in-degree of u by 1
            for u in graph.adjList[v]:
                graph.indegree[u] = graph.indegree[u] - 1

            # include current node in the path
            # and mark it as discovered
            path.append(v)
            discovered[v] = True

            # recur
            findAllTopologicalOrders(graph, path, discovered, N)

            # backtrack: reset in-degree
            # information for the current node
            for u in graph.adjList[v]:
                graph.indegree[u] = graph.indegree[u] + 1

            # backtrack: remove current node from the path and
```

```

        # mark it as undiscovered
        path.pop()
        discovered[v] = False

    # print the topological order if
    # all vertices are included in the path
    if len(path) == N:
        print(path)

# Print all topological orderings of a given DAG
def printAllTopologicalOrders(graph):

    # get number of nodes in the graph
    N = len(graph.adjList)

    # create an auxiliary space to keep track of whether vertex is discovered
    discovered = [False] * N

    # list to store the topological order
    path = []

    # find all topological ordering and print them
    findAllTopologicalOrders(graph, path, discovered, N)

# Driver code
if __name__ == '__main__':

    # List of graph edges as per above diagram
    edges = [(5, 2), (5, 0), (4, 0), (4, 1), (2, 3), (3, 1)]

    print("All Topological sorts")

    # Number of nodes in the graph
    N = 6

    # create a graph from edges
    graph = Graph(edges, N)

    # print all topological ordering of the graph
    printAllTopologicalOrders(graph)

```

This code is contributed by Priyadarshini Kumari

Time Complexity: $O(V!)$, Here V is the number of vertices, $V!$ is absolute worst case.
(worst case example - any graph with no edges at all)

Auxiliary Space: $O(V)$, for creating an additional array and recursive stack space.

Kahn's algorithm

from collections import deque

```

def topoSort(adj):
    n = len(adj)
    indegree = [0] * n
    res = []
    queue = deque()

    # Compute indegrees
    for i in range(n):
        for next_node in adj[i]:
            indegree[next_node] += 1

    # Add all nodes with indegree 0
    # into the queue
    for i in range(n):
        if indegree[i] == 0:
            queue.append(i)

    # Kahn's Algorithm
    while queue:
        top = queue.popleft()
        res.append(top)
        for next_node in adj[top]:
            indegree[next_node] -= 1
            if indegree[next_node] == 0:
                queue.append(next_node)

    return res

def addEdge(adj, u, v):

```

```

adj[u].append(v)

# Example
n = 6
adj = [[] for _ in range(n)]

addEdge(adj, 0, 1)
addEdge(adj, 1, 2)
addEdge(adj, 2, 3)
addEdge(adj, 4, 5)
addEdge(adj, 5, 1)
addEdge(adj, 5, 2)

res = topoSort(adj)
print(*res)

```

Time Complexity: $O(V+E)$. For Performing BFS
Auxiliary Space: $O(V)$.

SHORTEST PATH

Dijkstra's algorithm

Given a **weighted undirected** graph and a source vertex **src**. We need to find the shortest path distances from the source vertex to all other vertices in the graph.

Note: The given graph does not contain any negative edge.

Detailed Steps:

- Create a distance array `dist[]` of size `V` and initialize all values to infinity (∞) since no paths are known yet.
- Set the distance of the source vertex to 0 and insert it into the priority queue.
- While the priority queue is not empty, remove the vertex with the smallest distance value.
- Check: if the popped distance is greater than the recorded distance for this vertex(`dist[u]`), it means this vertex has already been processed with a smaller distance, so skip it and continue to the next iteration.
- For each neighbor `v` of `u`, check if the path through `u` gives a smaller distance than the current `dist[v]`.
If it does, update `dist[v] = dist[u] + edge weight(d)` and push `(dist[v], v)` into the priority queue.
- Continue this process until the priority queue becomes empty.
- Once done, the `dist[]` array will contain the shortest distance from the source to every vertex in the graph.

```

import heapq
import sys

def dijkstra(adj, src):

    V = len(adj)

    # Min-heap (priority queue) storing pairs of (distance, node)
    pq = []

    dist = [sys.maxsize] * V

    # Distance from source to itself is 0
    dist[src] = 0
    heapq.heappush(pq, (0, src))

    # Process the queue until all reachable vertices are finalized
    while pq:
        d, u = heapq.heappop(pq)

        # If this distance not the latest shortest one, skip it
        if d > dist[u]:
            continue

        # Explore all neighbors of the current vertex
        for v, w in adj[u]:

            # If we found a shorter path to v through u, update it
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))

    # Return the final shortest distances from the source
    return dist

```

```

if __name__ == "__main__":
    src = 0

    adj = [
        [(1, 4), (2, 8)],
        [(0, 4), (4, 6), (2, 3)],
        [(0, 8), (3, 2), (1, 3)],
        [(2, 2), (4, 10)],
        [(1, 6), (3, 10)]
    ]

    result = dijkstra(adj, src)
    print(*result)

```

Time Complexity: $O((V+E)*\log V)$, Where E is the number of edges and V is the number of vertices.

Auxiliary Space: $O(V+E)$

Bellman-Ford Algorithm

Given a weighted graph with **V** vertices and **E** edges, along with a source vertex **src**, the task is to compute the shortest distances from the source to all other vertices. If a vertex is unreachable from the source, its distance should be marked as **10⁸**. In the presence of a negative weight cycle, **return -1** to signify that shortest path calculations are not feasible.

Negative weight cycle:

A negative weight cycle is a cycle in a graph, whose sum of edge weights is negative. If you traverse the cycle, the total weight accumulated would be less than zero.

In the presence of negative weight cycle in the graph, the shortest path **doesn't exist** because with each traversal of the cycle shortest path keeps **decreasing**.

Principle of Relaxation of Edges

- Relaxation means **updating** the shortest distance to a node if a shorter path is found through another node. For an edge **(u, v)** with weight **w**:
 - If going through u gives a shorter path to v from the source node (i.e., **distance[v] > distance[u] + w**), we update the distance[v] as **distance[u] + w**.
- In the bellman-ford algorithm, this process is repeated **(V - 1)** times for all the edges.

Why Relaxing Edges (V - 1) times gives us Single Source Shortest Path?

A shortest path between two vertices can have at most **(V - 1)** edges. It is not possible to have a simple path with more than (V - 1) edges (otherwise it would form a cycle). Therefore, repeating the relaxation process (V - 1) times ensures that all possible paths between source and any other node have been covered.

Detection of a Negative Weight Cycle

- As we have discussed earlier that, we need (V - 1) relaxations of all the edges to achieve single source shortest path. If one **additional relaxation** (Vth) for any edge is possible, it indicates that some edges with overall negative weight have been traversed once more. This indicates the presence of a **negative weight cycle** in the graph.

Bellman-Ford is a **single source** shortest path algorithm. It effectively works in the cases of negative edges and is able to detect negative cycles as well. It works on the principle of **relaxation of the edges**.

```

def bellmanFord(V, edges, src):

    # Initially distance from source to all other vertices
    # is not known(Infinite) e.g. 1e8.
    dist = [100000000] * V
    dist[src] = 0

    # Relaxation of all the edges V times, not (V - 1) as we
    # need one additional relaxation to detect negative cycle
    for i in range(V):
        for edge in edges:
            u, v, wt = edge
            if dist[u] != 100000000 and dist[u] + wt < dist[v]:

                # If this is the Vth relaxation, then there
                # is a negative cycle
                if i == V - 1:
                    return [-1]

            # Update shortest distance to node v
            dist[v] = dist[u] + wt
    return dist

if __name__ == '__main__':
    V = 5
    edges = [[1, 3, 2], [4, 3, -1], [2, 4, 1], [1, 2, 1], [0, 1, 5]]

```

```
src = 0
ans = bellmanFord(V, edges, src)
print(' '.join(map(str, ans)))
```

MINIMUM SPANNING TREE PRIM ALGORITHM

This algorithm always starts with a single node and moves through several adjacent nodes, in order to explore all of the connected edges along the way.

- The algorithm starts with an empty spanning tree.
- The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, and the other set contains the vertices not yet included.
- At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.

- Create a set **mstSet** that keeps track of vertices already included in MST.
- Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE.
- Assign the key value as 0 for the first vertex so that it is picked first.
- While **mstSet** doesn't include all vertices
 - => Pick a vertex **u** that is not there in **mstSet** and has a minimum key value.
 - => Include **u** in the **mstSet**.
 - => Update the key value of all adjacent vertices of **u**. To update the key values, iterate through all adjacent vertices. For every adjacent vertex **v**, if the weight of edge **u-v** is less than the previous key value of **v**, update the key value as the weight of **u-v**.

```
# Library for INT_MAX
import sys
```

```
class Graph():
    def __init__(self, vertices):
        self.V = vertices
        self.graph = [[0 for column in range(vertices)]
                      for row in range(vertices)]

    # A utility function to print
    # the constructed MST stored in parent[]
    def printMST(self, parent):
        print("Edge \tWeight")
        for i in range(1, self.V):
            print(parent[i], "-", i, "\t", self.graph[parent[i]][i])

    # A utility function to find the vertex with
    # minimum distance value, from the set of vertices
    # not yet included in shortest path tree
    def minKey(self, key, mstSet):

        # Initialize min value
        min = sys.maxsize

        for v in range(self.V):
            if key[v] < min and mstSet[v] == False:
                min = key[v]
                min_index = v

        return min_index

    # Function to construct and print MST for a graph
    # represented using adjacency matrix representation
    def primMST(self):

        # Key values used to pick minimum weight edge in cut
        key = [sys.maxsize] * self.V

        parent = [None] * self.V # Array to store constructed MST

        # Make key 0 so that this vertex is picked as first vertex
        key[0] = 0

        mstSet = [False] * self.V

        parent[0] = -1 # First node is always the root of

        for cout in range(self.V):

            # Pick the minimum distance vertex from
            # the set of vertices not yet processed.
```

```

# u is always equal to src in first iteration
u = self.minKey(key, mstSet)

# Put the minimum distance vertex in
# the shortest path tree
mstSet[u] = True

# Update dist value of the adjacent vertices
# of the picked vertex only if the current
# distance is greater than new distance and
# the vertex is not in the shortest path tree
for v in range(self.V):

    # graph[u][v] is non zero only for adjacent vertices of u
    # mstSet[v] is false for vertices not yet included in MST
    # Update the key only if graph[u][v] is smaller than key[v]
    if self.graph[u][v] > 0 and mstSet[v] == False \
    and key[v] > self.graph[u][v]:
        key[v] = self.graph[u][v]
        parent[v] = u

self.printMST(parent)

if __name__ == '__main__':
    g = Graph(5)
    g.graph = [[0, 2, 0, 6, 0],
               [2, 0, 3, 8, 5],
               [0, 3, 0, 0, 7],
               [6, 8, 0, 0, 9],
               [0, 5, 7, 9, 0]]

    g.primMST()

```

Time Complexity: $O(V^2)$, As, we are using adjacency matrix, if the input graph is represented using an adjacency list, then the time complexity of Prim's algorithm can be reduced to $O((E+V) * \log V)$ with the help of a binary heap.

Auxiliary Space: $O(V)$

Efficient Implementation using Priority Queue and Adjacency List

For adjacency list representation, we can achieve $O((E+V)*\log(V))$ because we can find all adjacent of every vertex in $O(V + E)$ time and we can get minimum using priority queue in $O(\log V)$ time.

- We use a [priority queue](#) (min-heap) to always select the edge with the smallest weight.
- Push the first vertex and its weight into the queue.
- While the queue is not empty, extract the minimum-weight edge.
- If the vertex is unvisited, add its weight to a variable (res) and mark it as visited.
- Push all unvisited adjacent vertices of this vertex into the queue.
- After all vertices are processed, return the total weight stored in res.

```

import heapq

# Returns total weight of the Minimum Spanning Tree
def spanningTree(V, adj):

    # Min-heap storing (weight, vertex)
    pq = []
    visited = [False] * V
    res = 0

    # Start from node 0
    heapq.heappush(pq, (0, 0))

    while pq:

        wt, u = heapq.heappop(pq)

        if visited[u]:
            continue

        res += wt
        visited[u] = True

        # Push adjacent edges
        for v in adj[u]:
            if not visited[v[0]]:
                heapq.heappush(pq, (v[1], v[0]))

```



```

return res

if __name__ == '__main__':
    V = 3
    adj = [[] for _ in range(V)]

    adj[0].append([1, 5])
    adj[1].append([0, 5])

    adj[1].append([2, 3])
    adj[2].append([1, 3])

    adj[0].append([2, 1])
    adj[2].append([0, 1])

    print(spanningTree(V, adj))

```

KRUSKAL ALGORITHM

A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected, and undirected graph is a [spanning tree](#) (no cycles and connects all vertices) that has minimum weight. The weight of a spanning tree is the sum of all edges in the tree.

Below are the steps for finding MST using Kruskal's algorithm:

- Sort all the edges in a non-decreasing order of their weight.
- Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it. It uses the [Disjoint Sets](#) to detect cycles.
- Repeat step 2 until there are (V-1) edges in the spanning tree.

It picks the minimum weighted edge at first and the maximum weighted edge at last. Thus we can say that it makes a locally optimal choice in each step in order to find the optimal solution.

```

from functools import cmp_to_key

def comparator(a,b):
    return a[2] - b[2];

def kruskals_mst(V, edges):

    # Sort all edges
    edges = sorted(edges,key=cmp_to_key(comparator))

    # Traverse edges in sorted order
    dsu = DSU(V)
    cost = 0
    count = 0
    for x, y, w in edges:

        # Make sure that there is no cycle
        if dsu.find(x) != dsu.find(y):
            dsu.union(x, y)
            cost += w
            count += 1
            if count == V - 1:
                break
    return cost

# Disjoint set data structure
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [1] * n

    def find(self, i):
        if self.parent[i] != i:
            self.parent[i] = self.find(self.parent[i])
        return self.parent[i]

    def union(self, x, y):
        s1 = self.find(x)
        s2 = self.find(y)
        if s1 != s2:
            if self.rank[s1] < self.rank[s2]:
                self.parent[s1] = s2
            elif self.rank[s1] > self.rank[s2]:
                self.parent[s2] = s1
            else:
                self.parent[s2] = s1
                self.rank[s1] += 1

```

```
if __name__ == '__main__':
```

```
# An edge contains, weight, source and destination
edges = [[0, 1, 10], [1, 3, 15], [2, 3, 4], [2, 0, 6], [0, 3, 5]]
print(kruskals_mst(4, edges))
```

Time Complexity: $O(E * \log E)$ or $O(E * \log V)$

- Sorting of edges takes $O(E * \log E)$ time.
- After sorting, we iterate through all edges and apply the find-union algorithm. The find and union operations can take at most $O(\log V)$ time.
- So overall complexity is $O(E * \log E + E * \log V)$ time.
- The value of E can be at most $O(V^2)$, so $O(\log V)$ and $O(\log E)$ are the same. Therefore, the overall time complexity is $O(E * \log E)$ or $O(E * \log V)$

Auxiliary Space: $O(E+V)$, where V is the number of vertices and E is the number of edges in the graph.

Per capire la densità, dobbiamo guardare la relazione tra il numero di nodi (V) e il numero di archi (E):

- In un grafo non orientato senza cappi, il numero massimo di archi possibili è dato dalla formula:
$$\frac{V(V-1)}{2}$$
- Un grafo è considerato **denso** quando il numero di archi E è nell'ordine di $O(V^2)$, ovvero cresce in modo quadratico rispetto al numero dei nodi.
- Un grafo si dice **completo** (la massima densità possibile) quando *tutti* i nodi sono connessi a tutti gli altri nodi. In questo caso, la densità è pari a 1 .

Un esempio pratico per capire la differenza

- **Grafo Sparso (es. Una rete stradale o i collegamenti tra computer in una rete LAN):** Una città ha 100 incroci (nodi), ma ogni incrocio è collegato solo a 3 o 4 strade (archi). Anche se aumentano gli incroci, le strade per incrocio rimangono poche. E cresce in modo lineare rispetto a V ($E \approx V$).
- **Grafo Denso (es. Un social network ristretto o i voli tra i grandi hub mondiali):** Pensa a un gruppo di 20 amici su WhatsApp dove quasi tutti si conoscono e comunicano tra loro direttamente. Ci sono tantissimi legami attivi. Se i nodi sono 20, gli archi saranno vicini al massimo teorico di 190.

Perché questa distinzione è importante?

Nel mondo dell'informatica e degli algoritmi, sapere se un grafo è denso o sparso cambia completamente il modo in cui conviene gestirlo:

1. **Rappresentazione in memoria:** Per i grafi **densi** si usa quasi sempre la **matrice di adiacenza** (una tabella $V \times V$), perché è efficiente e non spreca spazio, dato che la maggior parte delle celle conterrà un arco. Per i grafi sparsi si preferisce la *lista di adiacenza*.
2. **Scelta degli algoritmi:** Algoritmi diversi performano meglio a seconda della densità. Ad esempio, per trovare il cammino minimo (Shortest Path):
 - Su grafi **sparsi**, l'algoritmo di Dijkstra implementato con i *min-heap* è molto veloce ($O(E \log V)$).
 - Su grafi **densi**, dove $E \approx V^2$, la versione classica di Dijkstra con array o l'algoritmo di Floyd-Warshall possono risultare scelte altrettanto valide o necessarie.